Thread Pool

Thread Pool · Cap concurrent thumbnail jobs · Build with Waqar

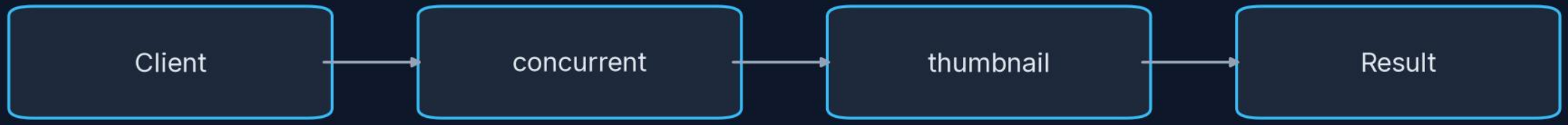
Today's Hook

- Problem: Cap concurrent thumbnail jobs
- Pattern: Thread Pool — earned from the pain, not trivia.
- Runnable main on JDK 25 with captured stdout.
- Naive → solution → failure paths → trade-offs.

The Problem

- Cap concurrent thumbnail jobs
- Naive design shows the cost clearly.
- Pattern isolates the force behind a clean boundary.
- Tests cover happy path and injected failures.

Structure



Thread Pool: Cap concurrent thumbnail jobs

Real-World Scenario

- Running example: Cap concurrent thumbnail jobs
- Same names in naive code, refactor, and Demo output.
- No shared StreamForge framing unless it fits.
- Product language first; pattern vocabulary second.

Naive Code

```java
package com.buildwithwaqar.patterns.concurrency.threadpool;

import com.buildwithwaqar.patterns.concurrency.threadpool.SimpleThreadPool;
import java.util.concurrent.atomic.AtomicInteger;
/** Problem: cap concurrent thumbnail jobs so the box is not fork-bombed. */
public final class UseCaseDemo {
  public static void main(String[] a) throws Exception {
    System.out.println("=== Thread Pool problem: thumbnail jobs ===");
    var n = new AtomicInteger();
    try (var pool = new SimpleThreadPool(3)) {
      for (int i=0;i<12;i++) pool.submit(n::incrementAndGet);
      Thread.sleep(250);
    }
    System.out.println("done=" + n.get());
  }
}
```

Why Naive Hurts

- Call sites couple to the wrong shape or leak on partial failure.
- Parameter drilling / duplicated envelopes / missing rollback.
- Hard to unit-test without the real dependency.
- New variants require shotgun edits.

Solution

```java
package com.buildwithwaqar.patterns.concurrency.threadpool;

import java.util.concurrent.BlockingQueue;
import java.util.concurrent.LinkedBlockingQueue;
import java.util.concurrent.atomic.AtomicBoolean;

/** Educational pool — prefer Executors / virtual threads in real code. */
public final class SimpleThreadPool implements AutoCloseable {
    private final BlockingQueue<Runnable> queue = new LinkedBlockingQueue<>();
    private final Thread[] workers;
    private final AtomicBoolean open = new AtomicBoolean(true);

    public SimpleThreadPool(int size) {
        workers = new Thread[size];
        for (int i = 0; i < size; i++) {
            workers[i] = Thread.ofVirtual().name("worker-" + i).start(() -> {
                while (open.get() || !queue.isEmpty()) {
                    try {
                        var task = queue.poll(50, java.util.concurrent.TimeUnit.MILLISECONDS);
                        if (task != null) task.run();
                    } catch (InterruptedException e) {
                        Thread.currentThread().interrupt();
                        return;
                    }
```

Solution (2)

```
                }
            });
        }
    }

    public void submit(Runnable task) {
        if (!open.get()) throw new IllegalStateException("closed");
        queue.add(task);
    }

    @Override
    public void close() {
        open.set(false);
        for (var w : workers) {
            try { w.join(1000); } catch (InterruptedException e) { Thread.currentThread().interrupt(); }
        }
    }
}
```

Runnable main

```
package com.buildwithwaqar.patterns.concurrency.threadpool;

import com.buildwithwaqar.patterns.concurrency.threadpool.SimpleThreadPool;
import java.util.concurrent.atomic.AtomicInteger;
/** Problem: cap concurrent thumbnail jobs so the box is not fork-bombed. */
public final class UseCaseDemo {
  public static void main(String[] a) throws Exception {
    System.out.println("=== Thread Pool problem: thumbnail jobs ===");
    var n = new AtomicInteger();
    try (var pool = new SimpleThreadPool(3)) {
      for (int i=0;i<12;i++) pool.submit(n::incrementAndGet);
      Thread.sleep(250);
    }
    System.out.println("done=" + n.get());
  }
}
```

Walkthrough Output

```
=== Thread Pool problem: thumbnail jobs ===
done=12
```

Step-by-Step Trace

- Each stdout line maps to a stage above.
- Re-run the main class locally on JDK 25.
- Failure-path lines prove compensations / rejects / missing files.
- Pause and predict the next line before revealing.

Modern Java 25 / Zazr

```
// Java 25 / Zazr direction
// records + sealed + switch
// inject Env / collaborators at the edge
// virtual threads for blocking fan-out
```

Trade-offs

- Clarity vs ceremony — refuse five types for two lines.
- Teaching demos omit production metrics/idempotency keys.
- Prefer simpler language features when they erase the force.
- Know the twin pattern often confused with Thread Pool.

Related Patterns

- Contrast Thread Pool with its neighbor pattern.
- Name the roles from memory in sixty seconds.
- Point to the season map for what comes next.
- Link the GitHub episode folder in the description.

Recap

- Problem → naive → Thread Pool → Demo stdout.
- Failure paths are part of the lesson.
- Teach when NOT to use it.
- Practice: break a test, fix it, re-run the main.